mobilen
news

volume 2 2023

İçindekiler

1 Merhaba **4**

2 Aklımda Kalanlar: Jake Wharton ile Röportaj **5**

2.1 Android'e Başlama Macerası . . . 5
2.2 Açık Kaynak Koda Bakışı . . . 7
2.3 JW'nin Google Macerası . . . 8
2.4 Zaman Yönetimi ve Çocuklu Hayat . . . 9
2.5 Teknik Yetiler, Yöneticilik ve Hobiler . . . 9
2.6 Kaynaklar . . . 10

3 Instagram'in Hikayesi **11**

3.1 Zawinski's Law . . . 11
3.2 Kaynaklar . . . 11

4 iOS SwiftUI Swipe Actions **12**

4.1 Swipe Actions nedir? . . . 12
4.2 SwipeActions Kullanımı . . . 13
4.3 Kaynak Kod Okuma: `PreferenceKey` kullanımı . . . 13
4.4 Extension methodlara Generic Constraint Uygulamak . . . 14
4.5 Kaynaklar . . . 14

5 Android Studio Bot **15**

5.1 Android Studio Bot Nedir? . . . 15
5.2 Android Studio Bot Nasıl Kurulur? . . . 16
5.3 Android Studio Bot'u Kullanmalı mıyız? . . . 16
5.4 Kaynaklar . . . 17

6 Kitap: Make Something Wonderful **18**

7 WWDC 23 Hakkında Bilmeniz Gerekenler **19**

7.1 Vision Pro . . . 19
7.2 Swift Macros . . . 20
7.3 SwiftUI Yenilikleri . . . 21
7.4 SwiftData . . . 22
7.5 Kaynakça . . . 23

8 Xcodes: Xcode'un Sürümlerini Yönetmenin Etkin Yolu **25**

8.1 Kaynaklar 27

1 Merhaba

Herkese iyi bayramlar :)

Bu sayımızda ünlü bir geliştiricinin macerasını, teknik bir component incelemesini, dünyanın en çok kullanılan uygulamalarından biri olan Instagram'ın ufak hikayesini bulabilirsiniz. Kullandığımız ve rastladığımız bazı geliştirme araçları hakkında linklerimizi yorumlarımızı da anlatmaya çalıştık.

Bu ay ayrıca herkesin merakla beklediği etkinliklerden Apple WWDC'yi geçirdik. Mobil dünyanın yeni üyesi VisionPro ürününün tanıtımı oldu. 'Spatial computing' gibi bir mecranin bu cihazların ucuzlaması sonrası önümüzdeki yıllarda daha çok gündemimizde olması olası.

Sizin de mobil konularda paylaşmak istedikleriniz var ise dergimizin Mobilen Github sayfasında bir `issue` oluşturup `markdown` olarak bize iletebilirsiniz.

Keyifli okumalar ;).

2 Aklımda Kalanlar: Jake Wharton ile Röportaj

Betül Sandıkçı yazdı.

Figure 1: Photo by Ian Schneider on Unsplash

Merhabalar sevgili Mobilen okuyucuları, bu sayı için yazacağım yazıma tamamen tesadüf eseri Jorge Castillo'nun Jake Wharton'la yaptığı röportajı okuyunca karar verdim.

Röportaja ilk denk geldiğimde bir an durdum açıp okumadan önce. Çünkü bir Android Developer olarak malum topluluk içinde bir efsanedir kendisi. Bazıları abartıldığını düşünürken bazıları da hakkında kilise içindeki İsa'dır ve tartışmasızdır demektedir. Ama herkesin üzerinde mutabık olduğu bir konu varsa o da Jake'in üretkenliğidir.

Topluluğu'u bir kenara bırakırsak onunla ilgili bildiklerim, duyumlarım, izlenimlerim neler? Bir çırpıda aklıma gelenleri özetlemek gerekirse; Androidle ilgili eksik ya da ağır işleyen ne varsa Jake'in düzenlemek için elini atıp yazdığı bir kütüphane var. Androidde güzel, kullanışlı ne varsa da Jake temellerini atmış ve sonunda Google da bu geliştirmeleri içeri almış, SDK'nın parçası haline getirmiş. Tüm bunların üstüne tabiki üretkenliği ve biraz da lafını sözünü sakınmayışı geliyor aklıma. Sakınmaması için sağlam gerekçeleri var ne de olsa :).

2.1 Android'e Başlama Macerası

JW ve Android hikayesi Google'ın 2007'de 10 milyon dolar ödüllü Android Developer Challange'ı açıklamasıyla başlıyor. JW daha üniversitenin henüz başında ve `challange`'ı kazanarak gerçek hayatta bir işte çalışmak zorunda olmamak için denemelere başlıyor. `Challange` için tasarladığı

uygulama bir çizim uygulaması. İşin içine girdikçe ve derinleştikçe kazın ayağının farklı olduğunu anlıyor. Bu 10 milyon basitçe kazanılabilecek bir miktar değil. Yeterince iyi olmadığını düşündüğü için projesini göndermekten vazgeçiyor. İyi ki bu tekrarlayan bir hastalık haline gelmemiş :D.

Android Geliştirici olmaya karar verene değin daldan dala atlamış, başka alanlar denemiş. Ama en sonunda Androidde devam etmeye karar vermiş. Karar verme sebebi benimle ve birçok güncel Android Geliştirici ile aynı. Mac alacak kadar parası yok, Microsoft ölüyor ve gelecek mobilde. Gayet rasyonel. Romantik bir gerekçesi yok. Çünkü hiç kimse bir sabah uykusundan Android Android Android diyerek ya da başka bir teknolojiyi sayıklayarak uyanmıyor. Seçimlerimiz güncel gerçeklerimize ve bu gerçeklerle örtüşen tutkularımıza bağlı olarak şekilleniyor.

JW'ın Android geliştici olma hikayesi tanıdık evet. Ama farklılaşan kısmı sonrası. JW'ın buraya kadarki hikayesinde beni düşündüren ve ibret aldığım şeyler var. Bir kez daha günceli takip etmenin önemi hemencik yakaladı beni. Sonunda bir ödül kazanılmayacak bile olsa bir müsabakaya girmek kendini, sınırlarını zorlamak çok önemli. Kaybetsen bile kazandıracak bir yöntem bu her şekilde. Tatmin edici bir deneyim. Burada Google'ın ve community'nin etkisinden de bahsetmek lazım. Sınırlarını zorlaman için birilerinin bir sınır koyması lazım doğal olarak önce. Community'ler ve teknoloji sahipleri de bu tür challangelar ile hem safları sıklaştırıyor hem teknolojilerini daha geniş çembere yayıyor hem de genç yeteneklerin ortaya çıkmasına vesile oluyor. Bence burada kökten uca kadar alınacak ibretler var. Çalışanların talepleri ve onları yönetenlerin vizyonu açısından. Takımlar arası müsabakalar keyifli, sıradışı ve eğlenceli olabilir mesela. Kim bilir eğer uslu çocuklar olurlarsa Şirinleri görmenin yanında nice JW'lar bile sıyrılabilir bu tür müsabakalarda.

Hikayede dikkatimi çeken noktalardan diğer birisi de başlangıç için bir çizim uygulaması yapmayı seçmiş olması. Yeni birisi başlarken sonucu görmek istiyor. Çok doğru bir yerden başlamış kanaatimce. Artık ne haddimeyse JW'a doğru başlamış yanlış başlamış demek :), Ama dedim gitti işte. Mobile özel o ekranı tanımak açısından, yaptıklarının direk sonucunu görmek açısından aşırı doğru bir yöntem bence. Ben de **Compose**'u öğrenirken iç içe kareler çizerek başlamıştım. İnsan neyi doğru yapıp yapmadığını somut olarak, uğraşmadan görmek istiyor. Deneme yanılma şeklinde öğrenmede denedikten sonra yanılıp yanılmadığını anlayana kadar geçen süre kısalıyor ve verimlilik artıyor böylece. Bir konuyu yeni öğrenenlere UI'da kısa yoldan tepki alabilecekleri şekilde başlatmak güzel bir yöntem olabilir ve genç yetenekler için özellikle uygulanabilir.

Jake ilk işine Android Developer olarak başlamıyor. Java miras kodlu bir server'da çalışarak başlıyor. Onun zamanında moda C# yazarak para kazanmak. Ama nedense talihin yardımıyla Java ile başlıyor mesleğe. Projeyi kurcalarken yeni şeyler keşfediyor. Projeyi **Git**'e geçiriyor. Projenin altyapısını farklı bir modele taşırken operasyonun her yönünü görüyor. Bu deneyimden sonra projenin altyapı görüntülemesini bir mobil uygulama üzerinden kontrol etme fikri aklına geliyor. Tam da bu anda ta taaa ilk açık kaynak kodlu kütüphanesini yayımlıyor ve bir Android Geliştiricisi

oluveriyor. Sonrasında meşhur şirket Square'e Android Geliştirici olarak geçiyor. Hikayesine ve üretmeye orada devam ediyor.

2.2 Açık Kaynak Koda Bakışı

Açık kaynak koda geliştirme yapmaya tamamen bencilce bir yerden başlıyor. Kendi ihtiyaçlarını karşılamak için proje üretiyor ve bunu Github'a yüklüyor. O zamanlar private repolar ücretli olduğu için de private yapmaya gerek duymuyor. Böylelikle kod paylaşma serüveni doğal olarak duygusal sebeplerle başlamış oluyor :D. Bir kütüphaneyi ya da aracı yazarken community'nin ya da başkalarının ihtiyaçlarını gözeterek ve daha iyi olsunlar diyerek başlamıyor. Niyetinde kimseyi memnun etmek de yok. Kendi yazdığı ve paylaştığı projelere başkalarının dahil olması, birlikte üretmek zevkli, keyifli, güzel ama motivasyon kaynağı bu olamaz düşüncesinde.

Bunalmamak ve motivasyon kaybına uğramamak için başkalarıyla gerektiğinde empati yapıyor gerektiğinde de onları görmezden geliyor. Motivasyonunu korumak için yaptığı şey çok basit. Temelde benim neye ihtiyacım var diye düşünmek ve ona göre hareket etmek. Sıfır noktasında düşüncelerle o kadar boğuşuyoruz ki başlamadan bırakıyoruz işlerimizi. Maskeyi önce kendimize takmalıyız.

Açık kaynak kodlu yazılım üretmenin en faydalı yanı başka insanlarla yardımlaşmak. Profesyonel hayatta çalışamayacağın, imkanın olmayacak insanlarla aynı projede yer almak çok tatmin edici. Takip ettiğin, imrendiğin belki de onların yerinde olmak istediğin insanlarla bir anlamda iş arkadaşı olma şansı sunuyor açık kaynak koda destek vermek. Minimumda başkalarına yardım etme ve faydalı olma duygusu yaşatıyor insana. Maksimumda ise profesyonel olarak sonsuz ihtimallerin kapısını açıyor. Bu aralık zannımca fena bir aralık değil bir geliştirici için.

Açık kaynak kodu geliştirirken Jake'in tavsiyesi kodu olabildiğince küçük ve odaklı tutmak. Her %15 fazla kod için aynı oranda geliştiriciye ihtiyaç var. Bu da bir süre sonra kar topu etkisi ile baş edilmez bir hale getiriyor projeyi. Herkesin ihtiyaçlarına kulak vermek ve ihtimallere göre geliştirme yapmak, sonrasında kodun sürdürülmesini imkansız hale getiriyor. Sürdürülemeyen kod da ölmeye mahkum oluyor. Çünkü bir süre sonra yapılması gereken hayati geliştirmeler bile yapılamayacak hale geliyor. Bu aslında sadece açık kaynak kod için değil herhangi bir yerde yazdığımız bir fonksiyon için de geçerli.

İhtiyacı belirleyip bunun dışına çıkmamak anahtar nokta. **Solid'** in S'i olan **Single Responsibility** prensibi burada devreye giriyor. Ancak bunu uygulayabilmek prensibin anlamını bilmekten elbette çok çok daha zor. İster açık kaynak ister küçük bir method ne olduğu farketmeksizin baştan amacını iyi belirlemek ve sonrasında buna sadık kalmak ve en önemlisi Hayır demenin gücünü kullanmak. Doğru yerde ve zamanda "*Hayır diyebilmek*" çok önemli ve işlerin karmaşıklaşmasını engelliyor. Memnun edicilik değil iş bitiricilik önemli oluyor günün sonunda.

2.3 JW'nin Google Macerası

Gelelim Jake'in kimilerine göre yarıda bıraktığı Google tecrübesine. Tabiki doğası gereği magazinsel bir mevzu olduğu için dikkat çekmesi normal. Benim anladığım Square'den Google'a geçerken bir yapılacaklar listesi varmış elinde. Kaldığı süre boyunca da bunların yapılması için domino etkisi yaratmayı başarmış. Ancak günün sonunda fazla bürokrasi ve teknik sınırlamalardan dolayı yılmış, usanmış. Geliştiriciye geliştirmek ile gerçek dünya problemlerine çözüm geliştirmek arasında kalınca da gerçek dünya problemlerini tercih etmiş. Bu benim yolum benim tercihim diyor. Square'deki macerasına kaldığı yerden devam etmeye karar vermiş ve geri dönmüş.

Verimli hissedemediği noktada bırakmaya karar vermesi oldukça ilham verici. Her ne kadar kolay gibi görünse de geldiğin yere geri dönmek cesaret gerektiren bir şey. Herşey bir kenara sadece egosal açıdan bile zor gelebilir. İçeride dışarıda olduğu kadar etkin olamadığını farkettiğinde gemiyi terketmeyi seçmiş. Ben mesela yapabilir miydim diye düşünüyorum? Bir yönüm çok yapılabilir dese de işin için de olduğumda bu kararı alırken oldukça zorlanacağımı, tüm cesaret toplamam gerekeceğini biliyorum. Neticede "skin in the game" diye bir şey var.

Jake'in CashApp'teki günleri aşırı planlı, düzenli, moda mod geçmiyormuş. Genellikle zamanının büyük kısmını açık kaynak koda ayırıyormuş. Kalan zamanda da küçük işleri toparlamak, PR'ları kontrol etmek, işleri adımlamakla geçiyormuş. Square açık kaynak kodu mesai dışı bir iş gibi görmüyor aksine bizatihi işin kendisi olarak görüyor ve destekliyor. Çünkü yeteneği içeri almak kadar onu tutmak, beslemek ve parlatmak da çok önemli. Takımlara ve yöneticilere düşecek bir hisse de bu olabilir belki.

Toplantı konusu teknik insanların profesyonel dünyada en sıkıcı bulduğu konuların başında geliyor. JW bu konuda kalabalık olanların seyrek olması en iyisi, bunun dışında bireysel bağlantılar daha verimli oluyor görüşünde çoğumuz gibi. Toplantı konusunda da *Solid*'in *S*'ini kullanmakta fayda var gibi :D .

İşin başındakilere en önemli tavsiyelerinden birisi ürünün vizyonuna kendinizi aşırı kaptırmayın. Böyle olursa kişi kendi başarı duygusunu ürünün iniş çıkışlarına bağlayabiliyor. Ancak önemli olan zorlu teknik sorunları bulmak ve çözmeye odaklanmak. Ürünler geçicidir oysa bu yetkinlikler kalıcıdır. Özetle Jake ürünü önemseyin ama yeteri kadar, bir dengesi olsun görüşünde. Odaklılık konusunda denge bulunması gerektiği ve bıçağını bileylemenin en önemlisi olduğuna katılıyorum. Senin bıçağın keskinse gelen her şeyi kesersin. Ama bıçağı kütmemek için de neyi ne kadar keseceğini bilmelisin ;) .

Üretkenliği korumak için Jake'in taktiklerinden birisi çok fazla çalışmaya gerek kalmadan yapılabilecek bir sürü kolay görevleri halletmek. Bu taktik zihinsel olarak gevşemeyi sağlarken, diğerleri gözünde de üretkenliğinizi korumayı sağlıyor . Aynı taktik tükenmişlik hissi yaşandıktan sonra ilerleme hissini yeniden kazanmaya yardımcı olmak için de kullanılabilir.

2.4 Zaman Yönetimi ve Çocuklu Hayat

Çocuklu hayata geçtikten sonra zamanın daha da önemli bir kaynak haline geldiğinden ve onu korumak ve boşa sarfetmemek için de elinden geleni yaptığından bahsediyor. Kimsenin bölemeyeceği, bildirimlerin kapalı olduğu büyük bloklu zaman dilimleri yaratmak, müsaitliğinin başkaları ile eşzamanlı olamayacağı konusunda beklentileri yönetmek, arta kalan yetişemediği küçük işlerde de başkalarına güvenmek; Jake'in hayatında uyguladığı bazı yöntemler. Yan projelerde çalışabilmek için az az küçük zaman parçalarını değerlendiriyormuş. Aralardaki küçük zaman dilimlerinde az az çalışarak işlere odaklanmak tek bir anda o işi bitirmeye çalışmaktan çok daha motive edici diyor.

Ayrıca zaman kısıtının onu çok daha verimli hale getirdiğini çünkü hangi projelerin zamanını aldığını ve bunların onu gerçekten zaman ayırmaya değip değmeyeceğini düşünmeye zorladığından bahsediyor. Sınırsız beyin gücü ve zaman verildiğinde, aklımıza gelen her yeni şeyin peşine düşeriz. Gerçekten önemli olanın ne olduğunu unuturuz. Ancak sabahları ve akşamları tamamen ortadan kalktığında ve iyi bir uyku ihtiyacı arttığında, o gün sahip olabileceğimiz 2 saatin değeri 10 kat artacaktır. Zamanımız kıymetli, doğru şekilde doğru yerlere harcamalıyız. Zaman en önemli maaliyet kalemidir.

2.5 Teknik Yetiler, Yöneticilik ve Hobiler

Röportajın kalan kısımlarında Jake Dagger 1 ve 2 arasındaki farktan, hobisi olan araba yarışlarından ve arabalardan, şu anda üzerinde çalıştığı ve onu heyecanlandıran projelerden, ileride yönetici olmayı düşünüp düşünmediğinden ve daha başka küçük küçük bir çok detaydan bahsediyor.

Jake kendisi hakkında Google Experts sayfasında; "Temiz sınırlara, iyi soyutlamalara ve sezgisel API'lere sahip açık kaynak kodlu kütüphaneler ve araçlar üzerinde çalışan Android Geliştiricisi. Bazen. Ayrıca ara sıra bu kütüphanaleri kullanan bir uygulama üzerinde çalışır." şeklinde tanımlamış. Kendisini tanımlama şekli ve röportaj birleşince çok güzel örtüşüyor zihnimde. Açık kaynak koda odaklanmış motive bir hayatı ve arta kalan zamanlarda çalıştığı CashApp diye bir gerçek hayat projesi var ve teknik blogu da faydali.

Nicedir bildiğim ama röportajdan sonra daha emin olduğum şey ise verimlilik ve motivasyon açısından Hayır demenin gücü. Bu yeti yoksa da öğrenilmeli. Neye hayır dediğimiz ve deme şeklimiz kariyer yolumuzu ve onun zorluklarını belirliyor neticede.

Son olarak da Jake'in bir tavsiyesi ile sonlandıralım yazımızı. Takıldığınız yerde mutlaka mola verin, yürüyüş yapın, çay-kahve için, film izleyin, sonra geri gelin bir daha uğraşın. Çünkü programlama yaratıcılık isteyen bir şeydir, zorlamayın.

2.6 Kaynaklar

Jack Wharton Röportajı

3 Instagram'in Hikayesi

Suat Karakuşoğlu yazdı.

Sosyal hayatımızın en önemli aktörlerinden birisi olan Instagram'ın geçmişine dair Burbn'in Instagram'a dönüşümü yazısına okudum.

Yazıyı okumanızı tavsiye ederim. Bu yazıda rastladığım bir konu `feature bloat` karmaşasına karşı yaklaşım.

Instagram'ın Foursquare gibi bir çok yeteneğin yanına harici yetenekleri de yapmaya çalıştığı zamanlardan yalnızca `fotoğraf yayınlama` + `filtre uygulama` + `like` edebilme yeteneklerine odaklanması sayesinde ürün tanımı daha oturaklı bir çerçeveye çekilmiş ve bu Instagram'ın hızlı yükselişinde önemli rol oynamış.

Bana Zawinski'nin aşırı özellikle şişen uygulamalar hakkındaki alaycı ifadesini hatırlattı:

3.1 Zawinski's Law

> "Every program attempts to expand until it can read mail. Those programs which cannot so expand are replaced by ones which can."
>
> "Her program email okuyucusu olana kadar gelişir, gelişmeyenler ise gelişenler ile değiştirilir."
> — Jamie Zawinski

Jamie Zawinski's Law bu kurala "Law of Software Envelopment" diyor. Bu tuzağa çok yakalanmadan ürünlerin karakteristiğinin ve sınırlarının çizilmesi gerekiyor. Burada işin içine elbette sunulan özelliklerin düzgün bir kullanıcı deneyim tasarımı ile kullanıcının karşısına çıkması gibi derin konulara doğru ilerliyor.

3.2 Kaynaklar

- Burbn'in Instagram'a dönüşümü
- Zawinski's law

4 iOS SwiftUI Swipe Actions

Suat Karakuşoğlu yazdı.

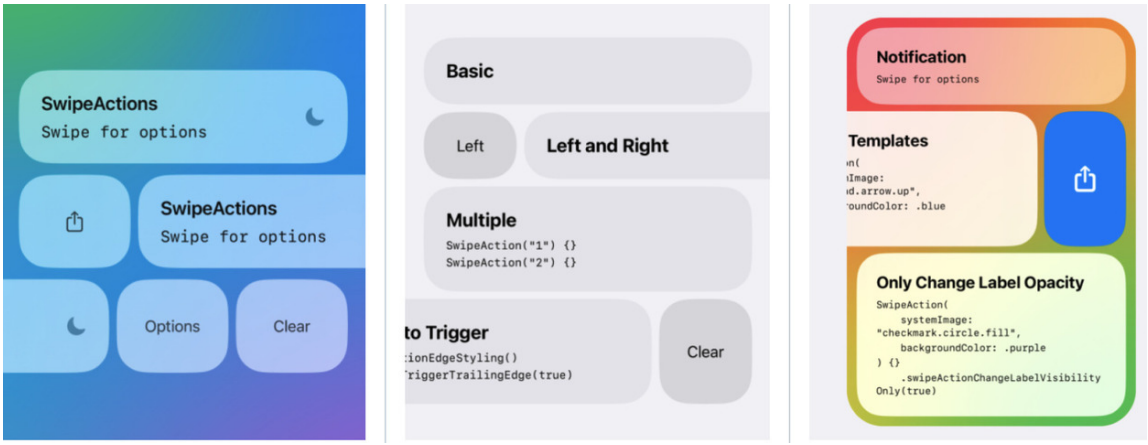

Figure 2: Örnek Swipe Actions

4.1 Swipe Actions nedir?

Merhabalar, herhangi bir ui elemanının üzerinden `swipe` ile çıkan aksiyonlar iOS kullanıcı deneyiminde önemli rol oynuyor.

Kullanıcı deneyimi açısından view'ın altında gizli, sağından veya solundan çıkan bu aksiyonlar görünmüyor olsa bile özellikle iOS kullanıcıları bu davranışı listelerde arayabiliyor.

Davranış olarak sağ ve sol demek yerine burada tercih edilen jargon `leading` ve `trailing`, bunun başlıca sebebi dillerin sağdan sola veya soldan sağa olarak 2 farklı şekilde yazılabilmesi. Soldan sağa yazılan dillerde sol-leading, sağ trailing iken, arapça-farsça gibi dillerde sol-trailing, sağ ise leading olmakta.

Swipe aksiyonları yazabilmek için yardımcı araçlar mevcut.

`Native` olarak SwiftUI tarafından iOS 15 ile SwiftUI List'elerine eklendi. iOS 15 SwiftUI SwipeActions adresinden native yöntem incelenebilir.

Ancak iOS-14 destekliyorsanız veya herhangi bir SwiftUI view'e swipe aksiyonları eklemek isterseniz bunun için güzel bir SwiftUI component'i açık kaynak olarak geliştirilip github'a `spm` destekli olarak koyulmuş.

Bu yazımızda Açık Kaynak SwipeActions Reposu'nu inceleyeceğiz.

4.2 SwipeActions Kullanımı

```
import SwiftUI
import SwipeActions

struct ContentView: View {
    var body: some View {
        SwipeView {
            Text("Hello")
              .frame(maxWidth: .infinity)
              .padding(.vertical, 32)
              .background(Color.blue.opacity(0.1))
              .cornerRadius(32)
        } trailingActions: { _ in
            SwipeAction("World") {
                print("Tapped!")
            }
        }
          .padding()
    }
}
```

4.3 Kaynak Kod Okuma: `PreferenceKey` kullanımı

Veri iletişimi view componentleri arasında Environment Objeler uzerinden `Parent` → `Child` view yönünde olabiliyor. Veya data binding ile `@Binding` çift taraflı `reactive` şekilde sağlanabiliyor.

PreferenceKey ile olan bu veri iletişiminde ise diğer bir ihtiyaç olan ise `child` → `parent` arasında veri geçişi yapabiliyoruz.

```
struct AllowSwipeToTriggerKey: PreferenceKey {
    static var defaultValue: Bool? = nil
    static func reduce(value: inout Bool?, nextValue: () -> Bool?)
    { value = nextValue() }
}
```

Preference key tanımlarken ihtiyaç duyulan `PreferenceKey` protocol'unu `conform` etmek. Bunun için bir varsayılan değer `defaultValue` gerekiyor. Ayrıca ikinci olarak `reduce` fonksiyonu, bu

fonksiyon dışarıdan sağlanan değerin preference olarak set edilmesine müdahale edilebilecek noktayı sağlıyor.

Kullanımı için 2 tane `modifier`'imiz mevcut: `preference` ve `onPreferenceChange`.

Preference Key ile veriyi yukarı aktarma burada gorebilirsiniz:

```
view.preference(key: AllowSwipeToTriggerKey.self, value: allowSwipeToTrigger)
```

Preference Key ile aşağıdan gelen veriyi burada okuma:

```
view.onPreferenceChange(AllowSwipeToTriggerKey.self) { allow in
    /// Unwrap the value first (if it's not the edge action, `allow` is `nil`).
    if let allow {
        swipeToTriggerLeadingEdge = allow
    }
}
```

Ayrıca apple sdk'inde de `navigationTitle` modifier'i ile `preference key` kullanılarak `child` üzerinden çağırılan bir method ile üst view'deki `title`'i değiştirebiliyor.

Özetle hiyerarşide yukarı veri taşıyabilmek için kullanılan bu yapının kullanıldığını görüyoruz. Daha detaylı anlamak için Magic of View Preferences in SwiftUI yazısına göz atabilirsiniz.

4.4 Extension methodlara Generic Constraint Uygulamak

`Generic` kullanarak `extension method`'larının yalnızca ilgili tipler ile kullanılmasını derleme seviyesinde sağlamak tatlı bir özellik. Burada ilgili örneği görebilirsiniz.

```
public extension SwipeAction where Label == Text, Background == Color {
    // Buraya yazılan methodlar yalnızca
    // Label Generic Parametresi Text olan ve Background'u Color olan 'SwipeAction'lara önerilir.
}
```

4.5 Kaynaklar

1. SwipeActions Repo
2. Apple PreferenceKey Dokümantasyonu
3. Magic of View Preferences in SwiftUI

5 Android Studio Bot

Tuğba Berk yazdı.

Google IO etkinliği, 10 Mayıs 2023 tarihinde gerçekleşti. Bu etkinlikte Google'ın yapay zeka desteğini mevcut araçlarına entegre ettiğini gördük. `Google Play Store`, `Google Haritalar`, `Google Mail` ve `Android Studio` gibi birçok araç bu yenilikler arasında yer aldı.

Yapay zekadaki bu gelişmeler, kod yazarken bize çeşitli faydalar sunmaya başladı bile. Bu yeni yapay zeka gelişmelerine ayak uydurabilen yazılımcılar olmak istiyorsak, Android Studio Bot'u hızlı bir şekilde öğrenmek önemli görünüyor. Kendi rollerimizi ve bu sistem içindeki araçlarımızı iyi anlamamız gerekebilir.

Peki, Android Studio Bot nedir? Kimler kullanır? Kullanmalı mıyız? Nasıl kuracağız? gibi sorulara cevap verelim.

5.1 Android Studio Bot Nedir?

Android Studio Bot bir yapay zeka asistanıdır.Android geliştiricilerin kod yazmalarına ve hatalarını çözmelerine yardımcı olmak için kullanabilecekleri bir araçtır. Android Studio Bot, Codey adlı yapay zeka asistanının AI yeteneklerini kullanarak geliştiricilere yardımcı olur. Codey, Google'ın temel dil modeli olan PaLM 2'yi kullanarak geliştirilen bir yapay zeka asistanıdır. Aslında, Google Colab ile çalışabilen bir bot diyebiliriz. Kısacası, Studio Bot, Codey'in sunduğu yetenekleri Android Studio ortamında daha kolay ve erişilebilir hale getirmiştir.

"ChatGPT" ve "Copilot" gibi araçlar mevcut olmasına rağmen, neden Studio Bot'a ihtiyaç duyacağımızı sorabilirsiniz. Benim cevabım şu: "Android Studio'dan çıkmadan kod araştırması yapmak ve sekme açmakla uğraşmamak daha pratik ve hoş olmaz mı?" Elbette, bu aracın sağladığı bilginin doğruluğu kullanmasında en önemli faktör olacaktır. Bu nedenle, doğruluğunu sağlamak için nasıl kullanmamız gerektiğini öğrenmeli ve deneyimlemeliyiz.

Google, Studio Bot kullanımını kolaylaştıran kısayollar hakkında bilgi verdi. Bu kısayollar sayesinde önemli bir zaman tasarrufu sağlanacağını düşünüyorum. İşte bazı örnekler:

- Tek tık ile kod ekleme

Studio Bot, oluşturduğu kodun yanında akıllı eylemler sunar. Bu eylemlerden biri, kodu hızlı bir şekilde açık editöre eklemenizi sağlar. Örneğin, bir internet izni kullanmanız gerektiği cevaplarda yer alıyorsa, "kod ekle" aksiyonunu tıkladığınızda Studio Bot, internet iznini manifest dosyasına ekler. Ayrıca, kodun nereye yazılması gerektiğini bilir, bu da bizi kopyala-yapıştır işlemlerinden kurtarır.

- Yardımcı kaynaklara erişmek

Belirli konular için, Studio Bot ilgili belgelendirme sayfasını açarak daha fazla bilgi sağlama seçeneği sunar. Size kaynak linklerine erişim sağlar, yani "oku, öğren, daha çok soru sor" gibi bir şey =)

- Bot'a kod'un içinden direkt soru sormak

Kod düzenleyicisinde metni seçip sağ tıklayarak ve `"Ask Studio Bot"` işlemini seçerek sohbet kutusunu seçilen kodla doldurabiliriz. Oradan, kodu gönderip göndermeme konusunda karar verebiliriz. Studio Bot, açık izniniz olmadan kod göndermeyecektir. "Ask Studio Bot" ayrıca kod düzenleyicisindeki uyarıları ve hataları gidermeniz için size yardımcı olabilir.

5.2 Android Studio Bot Nasıl Kurulur?

Studio Bot'u kurma ve kullanma konusunda bazı sorular sormaya başladığınızı düşünerek size üzücü bir haber vermek istiyorum. Şu anda Google, Studio Bot'u yalnızca ABD'de kullanıma sunmuştur. Bu nedenle, sadece ABD'deki kullanıcılar ücretsiz olarak erişebilirler.

Ancak, Studio Bot'u nasıl kuracağınızı bilmek isterseniz, aşağıdaki adımları takip edebilirsiniz:

1. İlk olarak, en son sürüm olan Android Studio Hedgehog | 2023.1.1 Canary 4'ü indirmeniz gerekmektedir.
2. Studio Bot'u daha etkin bir şekilde kullanmanız için veri paylaşma iznini onaylamanızı tavsiye ediyoruz. Bu izni etkinleştirmek için "Android Studio > Ayarlar > Görünüm ve Davranış > Sistem Ayarları > Veri Paylaşımı" yolunu izleyebilirsiniz.
3. Var olan bir projeyi açın veya yeni bir proje oluşturun. Ardından "Görünüm > Araç Pencereleri > Studio Bot" seçeneğine tıklayın.
4. Eğer istenirse, Google hesabınıza giriş yapın. Artık Studio Bot'u kullanmaya başlayabilirsiniz.

Bu şekilde Studio Bot'u kurabilir ve kullanabilirsiniz. Ancak unutmayın, şu anda sadece ABD'deki kullanıcılar ücretsiz olarak erişilebilmektedir.

5.3 Android Studio Bot'u Kullanmalı mıyız?

Gelin, Studio Bot'u kullanmalı mıyız konusuna değinelim. Şahsi fikrime göre, dünyada gelişen ve değişen şeylere katılmak ve bu dönüşümün bir parçası olmak, işimizin en keyifli yanıdır. Aslında, bu keyifli deneyimi kaçırmamak için hepinizi teşvik etmek isterim.

Ancak, burada dikkat etmemiz gereken önemli bir nokta var: Studio Bot, öğrenmekte olan bir çocuğun sizin elinizde büyüyeceğinin bilincinde olmanızı gerektirir ve şirketinizin özel bilgilerini bu

platform üzerinden paylaşmamalısınız. İyi olduğu kadar kötülerin de kod yazabileceğini unutmamamız önemlidir. Ve bu yeni dünyaya biraz daha kontrollü bir şekilde yaklaşmamız gerekmektedir.

Sonuç olarak, yapay zeka teknolojileri ile olan etkileşimimiz artıyor ve gelişiyor. Studio Bot gibi araçlar, geliştiricilerin kod yazma sürecini kolaylaştırmayı ve verimliliği artırmayı hedefliyor. Bu nedenle, yeni teknolojilere ve araçlara açık olmak, gelişmeleri takip etmek ve deneyimlemek önemlidir. Ancak, her aracın avantajlarını ve dezavantajlarını anlamak ve doğru bir şekilde kullanmak da gereklidir.

5.4 Kaynaklar

Android Studio Bot Preview

6 Kitap: Make Something Wonderful

Suat Karakuşoğlu yazdı.

Figure 3: Bilgisayar zihnin bisikletidir.

Steve Jobs'un aramızdan ayrılışının üzerinden 12 sene geçmiş :/.

İşlerinde ve ilişkilerinde hayatına dair daha derin bir bakış açısı yakalayabileceğimiz konuşmalarını, yazışmalarını ve hatıralarını biriktiren ve sanki kendisinin ağzından yazılmış bir kitap olarak ücretsiz yayınlanan bu kitabı zevkle okuyacağınıza inanıyorum.

İçerisindeki konulara değinmek gerekirse:

- Apple'dan ayrılışı, NeXT'in kuruluşu ve sonrasında tekrar Apple'a satılıp geri dönüşündeki hikayeler.
- Pixar'da elde ettiği başarılar.
- Teknolojiyi, sanatı, kullanıcı deneyimini iç içe görmesi ve kendi eğitim hayatındaki yaklaşımlarını bulabilirsiniz.

Apple Books üzerinden veya indirerek faydalanabilirsiniz

7 WWDC 23 Hakkında Bilmeniz Gerekenler

Alper Cem Öztürk yazdı.

Apple'ın WWDC'si geçtiğimiz günlerde gerçekleşti ve hem kullanıcılar hem de geliştiriciler için bir ton yeni özellik ve geliştirmeden bahsedildi. Hatta hepimizin yıllardır sızıntılardan takip ettiği tamamen yeni bir cihaz ve beraberinde işletim sistemi bile duyuruldu.

Bu yazıda, bir geliştiricinin bakış açısından bu yılın WWDC'sindeki favori güncellemelerimden bahsedeceğim. Bu yeniliklerle oynamak için çok fazla zamanım olmadığınından deneyimim çoğunlukla Keynote ve Platforms State of the Union sunumlarına dayandığını belirtmekte fayda var.

7.1 Vision Pro

Figure 4: Vision Pro

Etkinliğin en gözdesininden ilk bahsetmek en doğrusu olur diye düşünüyorum. Vision Pro, Apple'ın on yıldan uzun süredir geliştirilmekte olduğu artırılmış ve sanal gerçeklik cihazı. Çoğu kişi tarafından bu cihaz bir **headset** olarak isimlendirilse de Apple bu isimlendirmeyi kullanmaktan kaçınıyor. Bunun yerine sanal dünyayı gerçek dünya ile harmanlaması nedeniyle uzamsal bilgisayar (spatial computing) olarak adlandırıyor.

Apple, artırılmış gerçeklik deneyimi için etrafınızdakileri haritalandıran ve bunu sanal öğelerle zenginleştirilmiş dijital bir görüntüye çeviren kameralar kullanıyor. Sanal gerçeklik deneyimi için ise, bu kameraları kapatıp ve etrafınızda olup bitenlerden tamamen izole edilmiş gibi görünmenizi sağlayarak, yalnızca cihazın ekranlarında görüntülenenlere odaklanmanızı sağlıyor.

Vision Pro, Apple'ın cihaz için özel olarak tasarladığı bir işletim sistemi olan **visionOS**'u çalıştırıyor. VisionOS, kullanıcıyı çevreleyen alanda herhangi bir yere konumlandırılabilen uygulama pencereleriyle "sonsuz bir tuval" sağlamak üzere tasarlanmış.

Apple, Vision OS için tercih edilen geliştirme ortamının SwiftUI olduğu konusunda oldukça net bir tavır sergiledi. Elbette mevcut UIKit kodunuz kullanılabilecek ancak yeni başlangıçlar için en iyi seçim SwiftUI olacaktır.

Apple, SwiftUI desteğini daha çok arttırabilmek için Button, Toggle ve TabView gibi bir uygulamanın temel yapı taşlarından, Freeform'daki 3D panolar gibi yepyeni deneyimlere kadar tüm bunları ve çok daha fazlasını SwiftUI'ın desteklemesi için ayrıca çalıştığını belirtiyor.

VisionOS için daha derinlemesine bir SwiftUI deneyimine ihtiyaç duyuyorsanız Meet SwiftUI for Spatial Computing videosuna bir göz atmanızda fayda var.

7.2 Swift Macros

Hepimizi heyecanlandıran bir diğer haber ise Swift Macros'un duyurulmasıydı. Daha önceden zaten property wrappers ve function builders gibi tool'lara sahiptik. Bu tool'lar, arka plandaki mantığı kapsüllememize ve boilerplate kod miktarını azaltmamıza olanak sağlayan özelliklerdi. Swift 5.9 ile birlikte, toolkit'e makrolar da ekleniyor. Makrolar, daha fazla esneklik sağlayarak kod yazma sürecinde daha fazla kontrol ve kolaylık sunmayı vadediyor. Bu yeni özellik, Swift geliştiricilerinin daha temiz ve okunabilir kod yazmalarına önemli bir katkı sağlayacağa benziyor.

Swift makroları diğer tool'lardan oldukça farklı bir şekilde çalışır ve size farklı imkanlar sunar. Property wrappers'dan farklı olarak, Swift makroları derleme zamanında çalıştırılır. Bu da, kodunuzu derlemeye geçirmeden önce bile çalıştırarak hataları ve uyarıları tespit edebileceğiniz anlamına gelir. Bu özelliği sayesinde, Xcode'un sağladığı uyarıları kullanarak kodunuzu daha önceden doğrulayabilir ve olası hataları önleyebilirsiniz. Bu yönüyle, Swift makroları geliştiricilere daha güvenli ve hata ayıklamaya daha uygun bir ortam sunuyor.

Swift makroları ile alakalı daha detaylı inceleme yapmak isterseniz, Expand on Swift Macros'a bir göz atmanızda fayda var.

7.3 SwiftUI Yenilikleri

Her yıl SwiftUI yenilik ve iyileştirmelerine UIKit'in yerini tamamen alabilir mi gözüyle bakıyoruz. Her ne kadar buna daha zaman var gibi dursa da, bu yılki SwiftUI yenilikleri de bir hayli göze çarpıyor.

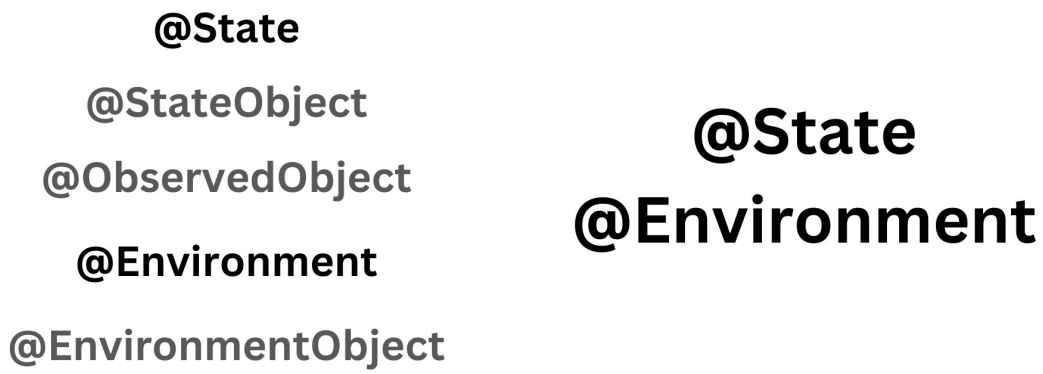

Figure 5: SwiftUI için yeni macrolar

Bu yılki SwiftUI değişiklikleri, veri akışını daha da sadeleştirmeyi hedefliyor. `StateObject`, `@ObservedObject` ve `@EnvironmentObject` property wrapper'ları `@State` ve `@Environment` altında toplanmış.

Ayrıca, artık nesne düzeyindeki `@Observable` macro sayesinde, `@Published` olarak parametreleri ayrı ayrı belirtmenize gerek kalmayacak.

Bir diğer büyük gelişmelerinden biri, animasyonlar için verilen desteğin arttırılmasıdır. Animasyonlar etkin bir şekilde yapıldığında, uygulamanın kalitesini arttırıyor. Günümüzde rekabetin bu kadar yüksek olduğu günlerde, kullanıcılarının uygulamaları kullanmaya devam etmesi için arayüz iyi görünmeli ve iyi hissettirmelidir. Bunun için artık keyframing ve animasyon aşamaları aracılığıyla animasyonlar üzerinde daha fazla kontrol sağlayabiliyoruz. Buna ek olarak spring modifier'ı ve animasyonlu SF symbols aracılığıyla oyun alanımız artık daha fazla genişletilmiş.

```
class Bird: ObservableObject {
  @Published var name: String
  @Published var species: Species
  @Published var songs: [Song]
}
```

```
@Observable
class Bird {
  var name: String
  var species: Species
  var songs: [Song]
}
```

Figure 6: SwiftUI örnek macro

SwiftUI, artık element dönüşümlerinde ayrı aşamaları tanımlamanıza ve zaman içinde animasyonun özellikleri değiştirmek için keyframe'ler oluşturmanıza olanak tanıyor. Güncellenmiş SF symbols paketi, artık kodla tetikleyebileceğiniz önceden tanımlanmış animasyonlarla birlikte geliyor. Bu sayede, SwiftUI'de animasyonları daha kolay ve esnek bir şekilde kontrol edebileceğiz.

Sonuç olarak, SwiftUI'ın popülerleşmesiyle birlikte, animasyon ve kullanıcı arayüzü tasarımı alanında daha fazla destek sunulmaya başlandı. Böylece, geliştiriciler uygulama kullanıcı deneyimini geliştirmek için daha fazla seçenek ve esnekliğe sahip olabilir. SwiftUI'nin bu güncellemeleri, geliştiricilerin iOS ve macOS uygulamalarında daha güçlü ve etkileyici kullanıcı arayüzleri oluşturmasına yardımcı olacağa benziyor.

SwiftUI yeniliklerine daha yakından göz atmak isterseniz What's new in SwiftUI videosunun bu konuda yardımı dokunabilir.

7.4 SwiftData

Core Data ile tanışanlar, onun öğrenilmesi ve kullanılması en kolay framework olmadığını bilirler. Core Data genellikle string literal'e dayanır. Derleme zamanı doğrulamasından yoksundur ve potansiyel yazım hatalarına yer bırakır. Bu durumda eğer bir hata yaparsanız, sadece uygulamanız

Figure 7: SwiftData

çöktüğünde bu hatayı öğrenebilirsiniz.

SwiftData, Core Data'ya alternatif olarak Swift ile oluşturulmuş ve Swift için tasarlanmış bir veritabanı çözümüdür. Bu, Swift'in kendisiyle uyumlu olan aynı prensipleri takip ettiği anlamına gelir. SwiftData hızlı, modern ve güvenlidir. Zaten aşina olduğunuz teknolojilere dayanmaktadır. SwiftData, modeller için ayrı dosyalar kullanmadığından, aynı veri yapılarını iki kez tanımlamanız gerekmez.

Apple, SwiftData'nın SwiftUI göz önünde bulundurularak tasarlandığını ve Core Data entegrasyonunun oldukça kolay olduğunu belirtiyor. Bunun yanı sıra Core Data ile aynı şekilde, SwiftData'nında **undo** ve **redo** gibi yaygın senaryoları desteklediğini söylemekte fayda var.

SwiftData'nın en önemli dezavantajlarından biri, yalnızca yeni işletim sistemi sürümlerinde mevcut olmasıdır. Bu nedenle, eski sürümleri desteklemeyi bırakmanız veya birkaç yıl beklemeniz gerekebilir, ancak bu kararın birçok kullanıcıyı etkileyeceğini bilerek hareket etmek en doğrusu olacaktır.

SwiftData ile ilgili daha derinlemesine bilgi sahibi olmak için Meet SwiftData videosuna göz atmak isteyebilirsiniz.

WWDC 23, Apple'ın teknoloji ve tasarım yaklaşımının sınırlarını genişletmeye devam ettiğini gösterdi. Vision Pro, SwiftUI'nin gelişmiş yetenekleri ve SwiftData gibi yenilikler, hem geliştiriciler hem de kullanıcılar için çığır açan değişiklikler getirecek gibi görünüyor. Her yıl olduğu gibi, Apple'ın sunduğu yenilikler, geliştiricilerin araçları ve teknolojileri kullanma şekillerini dönüştürecek. Bu da, kullanıcıların ürün ve hizmetleri deneyimlemelerinde yeni ve heyecan verici değişikliklere yol açmaya devam edecek.

Bu sayıda WWDC23 'ün önemli gelişmelerini sizlerle paylaştım. Umarım faydalı olmuştur. Sonraki sayıya kadar görüşmek üzere!

7.5 Kaynakça

1. Expand on Swift Macros

2. What's new in SwiftUI
3. Meet SwiftData

8 Xcodes: Xcode'un Sürümlerini Yönetmenin Etkin Yolu

Egehan Kalaycı yazdı.

Merhaba Mobilen okurları, bugün sizlere Xcode'un farklı sürümlerini nasıl kolaylıkla yönetebileceğinizi anlatacağım.

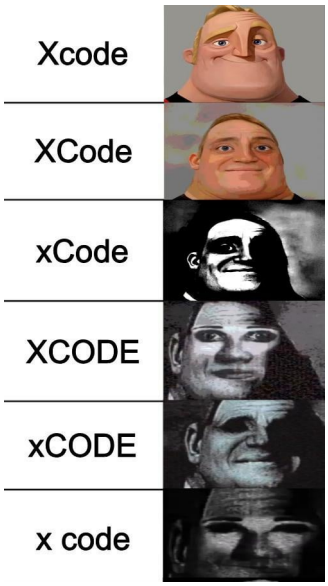

Figure 8: Xcode Naming

Öncelikle, **Xcode**'un doğru yazımı konusunda bir anlaşma yapalım. İnsanlar bazen XCode, xCode gibi tuhaf kombinasyonlarla karşımıza çıkabilirler. Ancak gerçek şu ki, doğru yazımı **Xcode**'dur. Evet, büyük harfle "**X**" ile başlayıp ardından "**code**" olarak devam eder. Şimdi, bu konuda anlaştıysak **Xcode**'un farklı sürümlerini yönetme konusuna odaklanalım.

Xcode, Apple platformlarında uygulama geliştirmek için kullanılan bir entegre geliştirme ortamıdır (IDE). Ancak, App Store'dan Xcode'u indirmenin bazı dezavantajları vardır. En güncel sürümleri bulmak veya yavaş bir indirme hızıyla karşılaşmakta sorun yaşayabilirsiniz. Daha da önemlisi, eski sürümlere geri dönmek veya birden fazla sürümü aynı anda kullanmak isterseniz, App Store bu özellikleri sunmamaktadır.

İşte tam da bu noktada size bir çözüm sunmak istiyorum: Xcodes!

Xcodes, Xcode'un farklı sürümlerini tek bir komutla indirmenizi sağlayan harika bir araçtır. Bu sayede istediğiniz sürümü, yüksek indirme hızlarıyla elde edebilirsiniz. Xcodes, indirme işlemlerini gerçekleştirmek için aria2 adlı bir komut satırı aracını kullanır. Bu araç, indirme hızınızı optimize

etmek için çoklu bağlantılar ve paralel indirmeler kullanır. Böylece Xcode sürümlerini daha hızlı bir şekilde bilgisayarınıza indirebilirsiniz.

Gelelim `Xcodes`'u kullanmaya. Bunun iki farklı yöntemi bulunmaktadır.

İlk yöntem, Xcodes'un kullanıcı dostu bir arayüzü olan native uygulamasını indirip kullanmaktır. Bu sayede sürümleri görsel olarak kolaylıkla yönetebilirsiniz.

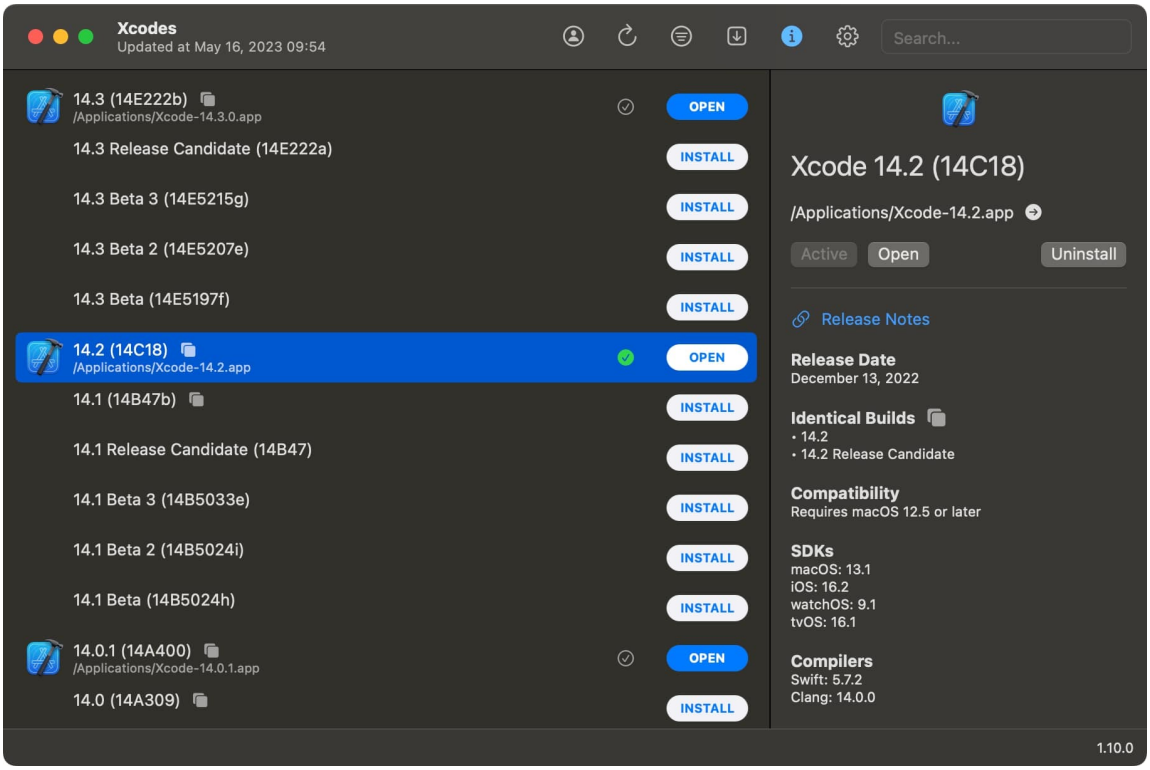

Figure 9: Xcodes Gui

İkinci yöntem ise terminal üzerinden Xcodes'u kullanmaktır. Eğer terminal üzerinden kullanmak isterseniz, `xcodes` komutunu kullanarak çeşitli işlemleri gerçekleştirebilirsiniz. Örneğin, mevcut sürümleri görüntülemek için `xcodes list` komutunu kullanabilir, indirmek istediğiniz sürümün yanındaki numarayı yazarak indirme işlemini başlatabilirsiniz. Ayrıca, indirdiğiniz sürümleri `xcodes installed` komutu ile görüntüleyebilir ve `xcodes select` komutuyla varsayılan sürümü belirleyebilirsiniz.

Artık Xcode'un farklı sürümlerini kolaylıkla yönetebilir, istediğiniz sürüme hızlıca erişebilir ve projelerinizde esnek bir şekilde çalışabilirsiniz. Bir sonraki yazıda tekrar buluşmak dileğiyle.

```
$ xcodes install 12.0
(1/6) Downloading Xcode 12.0.0: 100%
(2/6) Unarchiving Xcode (This can take a while)
(3/6) Moving Xcode to /Applications/Xcode-12.0.0.app
(4/6) Moving Xcode archive Xcode-12.0.0.xip to the Trash
(5/6) Checking security assessment and code signing
(6/6) Finishing installation
xcodes requires superuser privileges in order to finish installation.
macOS User Password:

Xcode 12.0.0 has been installed to /Applications/Xcode-12.0.0.app
```

Figure 10: Xcodes Terminal

8.1 Kaynaklar

1. https://www.xcodes.app/
2. https://github.com/XcodesOrg